

RESENHA CRÍTICA

PARNAS, D. L. On the Criteria To Be Used in Decomposing Systems into Modules

No artigo “On the Criteria To Be Used in Decomposing Systems into Modules”, publicado em 1972, David L. Parnas apresenta uma reflexão decisiva para a engenharia de software ao discutir quais critérios devem orientar a decomposição de sistemas em módulos. Em vez de tratar a modularização apenas como uma técnica de organização do código, o autor a analisa como um princípio de projeto capaz de melhorar a flexibilidade, a compreensibilidade e a manutenção de sistemas complexos. O texto parte de uma constatação importante: embora a modularização já fosse reconhecida como desejável, havia pouca discussão sobre o critério adotado para dividir um sistema. A contribuição central de Parnas consiste em mostrar que diferentes formas de modularizar produzem consequências muito diferentes para a evolução do software.

Para desenvolver sua argumentação, o autor utiliza o exemplo de um sistema de produção de índice KWIC, cuja função é receber um conjunto de linhas, gerar todos os deslocamentos circulares dessas linhas e apresentá-los em ordem alfabética. A partir desse problema, Parnas compara duas propostas de decomposição. A primeira representa a abordagem mais convencional, pois divide o sistema de acordo com as etapas do processamento: entrada, deslocamento circular, ordenação alfabética, saída e controle principal. Essa estrutura parece natural porque acompanha o fluxo de execução do programa. Entretanto, o autor demonstra que tal escolha faz com que decisões importantes de projeto, como o formato de armazenamento e os detalhes de acesso aos dados, fiquem espalhadas por vários módulos, criando forte acoplamento entre eles.

A segunda decomposição apresentada por Parnas segue um critério menos intuitivo, mas muito mais robusto: cada módulo deve esconder uma decisão de projeto específica. Nessa proposta, há módulos voltados ao armazenamento de linhas, à entrada, ao deslocamento circular, à alfabetização, à saída e ao controle, porém a diferença está no fato de que cada módulo expõe apenas uma interface abstrata e oculta sua implementação interna. Esse princípio, mais tarde consagrado como *information hiding*, faz com que alterações em aspectos como formato de armazenamento, estratégia de ordenação ou modo de gerar os deslocamentos circulem menos pelo sistema. Em outras palavras, a mudança deixa de contaminar vários componentes e tende a ficar concentrada no módulo responsável pela decisão escondida.

A tese defendida no artigo é particularmente relevante porque desloca o foco da simples divisão funcional para a proteção das decisões sujeitas a mudança. Parnas argumenta que modularizar conforme um fluxograma é inadequado para sistemas maiores, pois o fluxo temporal da execução não coincide necessariamente com a melhor organização estrutural do software. Ao contrário, o projeto deve começar identificando decisões difíceis, voláteis ou críticas, para então isolá-las. O ganho dessa estratégia aparece em três dimensões destacadas pelo autor: desenvolvimento independente, facilidade de compreensão e maior capacidade de adaptação do produto. Quando os módulos se comunicam por interfaces mais simples e abstratas, diferentes equipes podem trabalhar com menos dependência mútua, e o sistema torna-se mais inteligível para quem precisa estudá-lo ou modificá-lo.

Outro aspecto importante do texto é que Parnas não apresenta sua proposta de forma ingênua. Ele reconhece que a decomposição baseada em ocultação de informação pode introduzir custos de eficiência se for implementada de maneira descuidada, principalmente pelo excesso de chamadas entre módulos. Ainda assim, em vez de abandonar a proposta, o autor sugere que a implementação precisa evoluir para preservar os benefícios estruturais sem sacrificar o desempenho. Essa observação revela a maturidade do argumento: o critério de projeto não é justificado apenas por elegância teórica, mas por sua utilidade prática diante das mudanças inevitáveis do desenvolvimento de software.

Do ponto de vista crítico, o artigo possui grande força conceitual, sobretudo pela clareza com que contrapõe duas formas de pensar o projeto de sistemas. O exemplo do KWIC, embora simples, é extremamente didático e suficiente para evidenciar por que um sistema aparentemente bem dividido pode, na verdade, ser rígido e difícil de manter. Além disso, o texto antecipa discussões que se tornariam centrais em áreas como orientação a objetos, encapsulamento, arquitetura de software e design de interfaces. Seu principal ponto forte está justamente nessa capacidade de formular um princípio geral a partir de um caso pequeno, mostrando que decisões de projeto devem ser avaliadas segundo seu impacto nas mudanças futuras.

Como limitação, pode-se dizer que o artigo exige do leitor certa familiaridade com conceitos de programação e estrutura de sistemas, o que torna sua leitura menos imediata para iniciantes. Além disso, por ter sido escrito em um contexto técnico dos anos 1970, alguns exemplos de implementação soam datados em comparação com linguagens e ferramentas atuais. Mesmo assim, essa distância histórica não reduz a relevância do texto; ao contrário, evidencia o

quanto a proposta de Parnas permaneceu atual. Em um cenário contemporâneo marcado por manutenção contínua, evolução de requisitos e integração entre múltiplos componentes, a ideia de esconder decisões de projeto continua sendo uma das bases do bom desenvolvimento de software.

Em síntese, a leitura do artigo permite compreender que modularização não é simplesmente quebrar um sistema em partes menores, mas escolher criteriosamente quais responsabilidades devem ser isoladas para reduzir o impacto das mudanças. Parnas demonstra, com rigor e objetividade, que a qualidade de uma decomposição depende menos da sequência visível do processamento e mais da forma como o projeto protege decisões internas. Trata-se de um texto clássico, ainda essencial, para quem deseja entender os fundamentos da engenharia de software e projetar sistemas mais flexíveis, compreensíveis e duráveis.